

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

ICS1512

Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Academic Year	2026–2027 (Odd)
Batch	2024–2029

Experiment 6

Bagging, Boosting, and Stacked Ensemble Models¹

Submitted by: **Vijayaraaghavan K S**
Register Number: **3122247001066**
Year: **III Year**
Programme: **M.Tech Integrated, CSE**
Faculty: **Dr. Poreddy Ajay Kumar Reddy**

¹GitHub Repository: <https://github.com/VijayaraaghavanKS/ML>

Contents

1	Aim and Objective	2
2	Problem Statement	2
3	Theory	2
3.1	Bagging (Bootstrap Aggregation)	2
3.2	Boosting	2
3.3	Stacked Ensemble (Stacking)	2
4	Machine Learning Algorithm	3
4.1	Bagging Classifier	3
4.2	Boosting (AdaBoost and Gradient Boosting)	3
4.3	Stacked Ensemble	3
5	Dataset	4
6	Preprocessing and Exploratory Data Analysis	4
7	Experimental Setup	5
8	Baseline Reference Model	5
9	Bagging	5
9.1	Baseline Performance	5
9.2	Hyperparameter Tuning	5
10	Boosting	6
10.1	Baseline Performance	6
10.2	Hyperparameter Tuning	7
11	Stacked Ensemble	8
11.1	Base Learner Performance	8
11.2	Base-Model Combination Search	8
12	Hyperparameter Tuning Summary	9
13	Cross-Validation Results	10
14	Final Comparison	10
15	Bias-Variance Analysis	11
16	Comparative Analysis	12
16.1	Observations	12
17	Observation Questions	13
18	Discussion	14
19	Conclusion	15
20	References	15

1 Aim and Objective

To implement three ensemble learning strategies – Bagging, Boosting, and a heterogeneous Stacked Ensemble – on the Wisconsin Diagnostic Breast Cancer dataset, tune each with 5-fold cross-validated hyperparameter search, and analyze how each strategy trades off bias against variance relative to a single Decision Tree.

2 Problem Statement

The input is the same 30-dimensional vector of nuclear-morphology measurements used in Experiment 5 (radius, texture, perimeter, area, smoothness, and related shape descriptors, each summarized as a mean, standard error, and “worst” value); the output is a binary diagnosis, malignant or benign. Experiment 5 already showed that a single tuned Decision Tree and a Random Forest tie on test-set accuracy while the forest wins on cross-validated accuracy and stability. This experiment asks a broader question: given the same base learner (or a mix of different ones), which combination strategy – parallel averaging (Bagging), sequential error correction (Boosting), or learned combination (Stacking) – makes the best use of it, and does the answer differ depending on whether the goal is reducing variance, reducing bias, or exploiting model diversity?

3 Theory

3.1 Bagging (Bootstrap Aggregation)

Bagging trains many copies of a high-variance base model, each on an independent bootstrap resample of the training data, and combines their predictions by majority vote. Because the resamples differ, the individual models’ errors are only weakly correlated, so averaging them cancels out a large share of their variance without changing their bias much. Bagging is most useful when the base learner is already low-bias but high-variance, which is exactly the profile of an unconstrained Decision Tree.

3.2 Boosting

Boosting trains base learners sequentially rather than independently: each new learner is fit to pay more attention to the samples the ensemble has gotten wrong so far (AdaBoost reweights misclassified samples before fitting the next learner; Gradient Boosting instead fits the next tree to the residual gradient of the loss function). This directly targets bias – a boosted ensemble of weak learners can represent a far more complex decision boundary than any one of them individually – though pushed too far (too many rounds, too high a learning rate) boosting can start to raise variance and overfit.

3.3 Stacked Ensemble (Stacking)

Stacking trains several heterogeneous base learners and a separate meta-learner that is trained on the base learners’ out-of-fold predictions to learn how to best combine them. Unlike bagging and boosting, which typically use one type of base learner, stacking is explicitly designed to exploit diversity: base learners with different inductive biases (a margin-based SVM, a probabilistic Naive Bayes classifier, and an axis-aligned Decision Tree, in this experiment) tend to make different, less-correlated mistakes, giving the meta-learner more complementary signal to work with than a homogeneous ensemble would.

4 Machine Learning Algorithm

4.1 Bagging Classifier

Working principle: draw `n_estimators` bootstrap samples (sampled with replacement, each of size `max_samples` $\times$ training set) from the training data, fit an independent copy of the base estimator (a Decision Tree here) on each sample, restricting each fit to a random subset of `max_features` columns, and combine the predictions by majority vote. **Mathematical formulation:** for B bootstrap models $T_1, \dots, T_B$, the ensemble prediction is $\hat{y} = \text{mode}\{T_1(x), \dots, T_B(x)\}$; since each T_b is trained on an independent resample, $\text{Var}(\hat{y}) \approx \text{Var}(T_b)/B$ under the (optimistic but directionally correct) assumption of independent errors, which is why more estimators generally reduces variance up to a point of diminishing returns. **Advantages:** straightforward to parallelize since every base model trains independently, and it reliably reduces variance for any high-variance base learner without requiring any change to how that base learner is trained. **Limitations:** it cannot fix a base learner that is systematically biased (averaging many equally wrong models does not make them right), and it loses the single-model's interpretability.

4.2 Boosting (AdaBoost and Gradient Boosting)

Working principle: AdaBoost starts with uniform sample weights, fits a weak learner, increases the weight of misclassified samples, and repeats, combining all learners in a weighted vote; Gradient Boosting instead fits each new tree to the negative gradient of the loss with respect to the current ensemble's predictions, effectively doing gradient descent in function space. **Mathematical formulation:** AdaBoost's final prediction is $\hat{y} = \text{sign}\left(\sum_{m=1}^M \alpha_m h_m(x)\right)$ where $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$ weights each weak learner h_m by its own weighted error rate ϵ_m ; Gradient Boosting builds $F_M(x) = F_0(x) + \sum_{m=1}^M \nu \cdot h_m(x)$ where each h_m is fit to the residual gradient $-\partial L / \partial F_{m-1}$ and ν is the learning rate controlling each step's contribution. **Advantages:** directly and iteratively reduces bias, so a boosted ensemble of shallow trees can fit decision boundaries a single shallow tree cannot; Gradient Boosting in particular tends to be very accurate on structured/tabular data such as this one. **Limitations:** training is inherently sequential (harder to parallelize than bagging), it is more sensitive to noisy labels and outliers since it keeps re-emphasizing hard-to-fit samples, and both `n_estimators` and `learning_rate` must be tuned jointly to avoid overfitting.

4.3 Stacked Ensemble

Working principle: fit each base learner on the full training set for final predictions, but generate the meta-learner's training data via cross-validated out-of-fold predictions from each base learner (so the meta-learner never sees a base learner's prediction on data that base learner was trained on), then fit the meta-learner (Logistic Regression here) on those out-of-fold predictions to learn how to weight and combine them. **Mathematical formulation:** with base learners $f_1, \dots, f_K$ producing out-of-fold predictions $\hat{p}_k(x)$, the meta-learner learns $\hat{y} = g(\hat{p}_1(x), \dots, \hat{p}_K(x))$ where g is a Logistic Regression over the K -dimensional space of base-learner outputs. **Advantages:** can outperform every individual base learner if their errors are sufficiently uncorrelated, since the meta-learner effectively learns a soft, data-driven voting scheme rather than a fixed rule like majority vote. **Limitations:** more expensive to train (every base learner is cross-validated internally before the meta-learner is fit), harder to interpret than any single base model, and offers little benefit if the chosen base learners are not actually diverse in how they err.

5 Dataset

This experiment reuses the same Wisconsin Diagnostic Breast Cancer dataset as Experiment 5 (569 samples, 30 numeric features, binary `diagnosis` label), loaded through scikit-learn’s `load_breast_cancer` function and written out as a CSV so it flows through the same shared EDA pipeline used by Experiments 1–5. After the EDA pipeline’s 80/20 stratified split, the working data is 455 training rows and 114 test rows across 30 features, with 357 benign and 212 malignant cases overall, and malignant encoded as the positive class throughout for the same clinical reason as before: a false negative (calling a malignant tumor benign) is the costlier error.

6 Preprocessing and Exploratory Data Analysis

The EDA pipeline reports zero missing values and zero duplicate rows across all 569 samples, so no imputation or de-duplication was needed. Every feature is numeric and was standardized before modelling. As in Experiment 5, 44 feature pairs exceed a 0.80 Pearson correlation, driven by the same near-deterministic relationships between radius, perimeter, and area at the mean/error/worst level – this redundancy does not meaningfully affect any of the three ensemble families used here, since tree-based and margin-based base learners alike only need one representative of a correlated group to make an equally good decision.

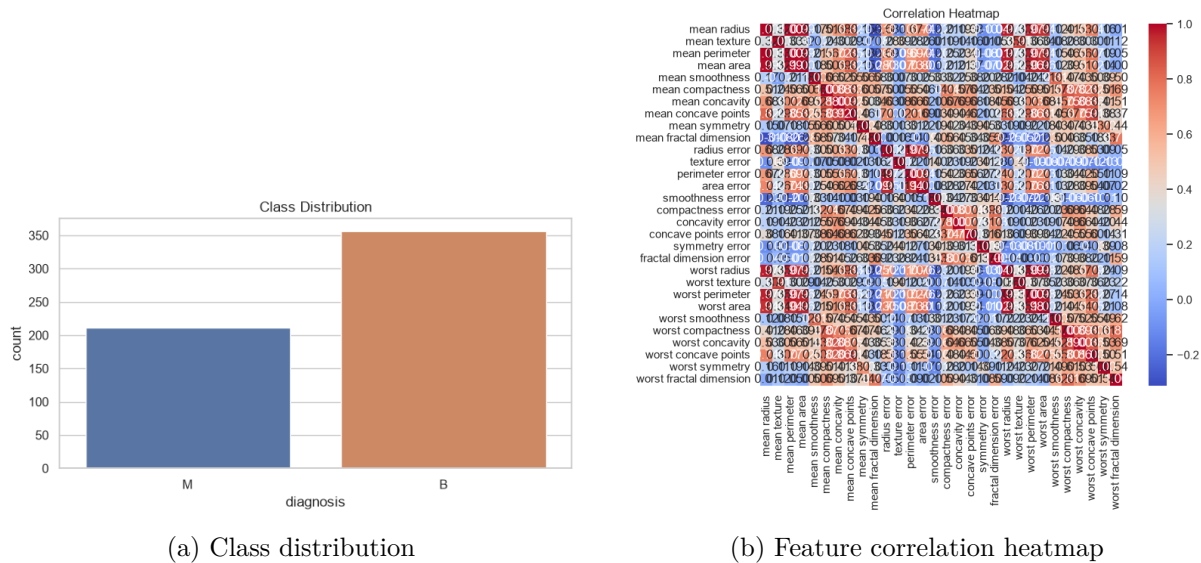


Figure 1: EDA diagnostics, reused from the shared pipeline.

Feature	SelectKBest score	p-value	Mutual information
worst radius	692.86	2.5×10^{-93}	0.4612
worst perimeter	717.25	2.1×10^{-95}	0.4546
worst area	522.19	1.9×10^{-77}	0.4533
mean concave points	684.53	1.3×10^{-92}	0.4460
worst concave points	733.73	8.8×10^{-97}	0.4329
mean perimeter	548.41	4.7×10^{-80}	0.4093

Table 1: Top features by mutual information with the diagnosis label.

The same size and shape descriptors that dominated Experiment 5’s tree-importance rankings top this list again, which is expected since the dataset and preprocessing are unchanged –

the interesting question in this experiment is not which features matter but which combination strategy makes the most of them.

7 Experimental Setup

Python Version	3.14.5
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	macOS (Darwin 25.5.0, arm64)
Hardware	Apple Silicon (arm64), local machine

Table 2: Software and hardware environment used to run this experiment.

8 Baseline Reference Model

Before evaluating the three ensembles, a single unconstrained Decision Tree was trained as a bias/variance reference point: test Accuracy=0.9298, F1=0.9048, and a 5-fold CV accuracy of mean=0.9011 with standard deviation 0.0326. Every ensemble below is compared back against this single-tree baseline to quantify what each combination strategy actually bought.

9 Bagging

9.1 Baseline Performance

Model	Accuracy	Precision	Recall	F1	Train Time (s)
Bagging (baseline)	0.9649	1.0000	0.9048	0.9500	0.0193
Bagging (tuned)	0.9649	1.0000	0.9048	0.9500	0.0420

Table 3: Baseline (10 default trees) vs. tuned Bagging on the test set.

Both the untuned and tuned Bagging ensembles land on exactly the same test-set metrics, which is not too surprising given how small the 114-row test set is and how strong the untuned default (10 bootstrap trees over a Decision Tree base) already was. Where tuning does show its value is in the ROC-AUC, which climbs from 0.9767 to 0.9917 – the tuned ensemble’s probability estimates are meaningfully better ranked even where the hard classification decisions match.

9.2 Hyperparameter Tuning

```

1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE
2 )
3 bagging_param_grid = {
4     "n_estimators": [10, 50, 100, 200],
5     "max_samples": [0.5, 0.7, 1.0],
6     "max_features": [0.5, 0.7, 1.0],
7 }
8 bagging_grid_search = GridSearchCV(
9     estimator=BaggingClassifier(estimator=DecisionTreeClassifier(
10         random_state=RANDOM_STATE), random_state=RANDOM_STATE),

```

```

9 param_grid=bagging_param_grid,
10 scoring={"accuracy": "accuracy", "f1": f1_scorer},
11 refit="accuracy", cv=cv, n_jobs=-1,
12 )

```

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.7	97.36	0.9643
50	1.0	96.92	0.9583
50	0.5	96.70	0.9542
100	0.7	96.70	0.9547
100	1.0	96.70	0.9551
200	1.0	96.70	0.9551
200	0.7	96.48	0.9519
10	0.5	96.26	0.9477

Table 4: Bagging hyperparameter evaluation using 5-fold cross-validation (best score per n_estimators/max_samples combination, top 8 of 12 shown).

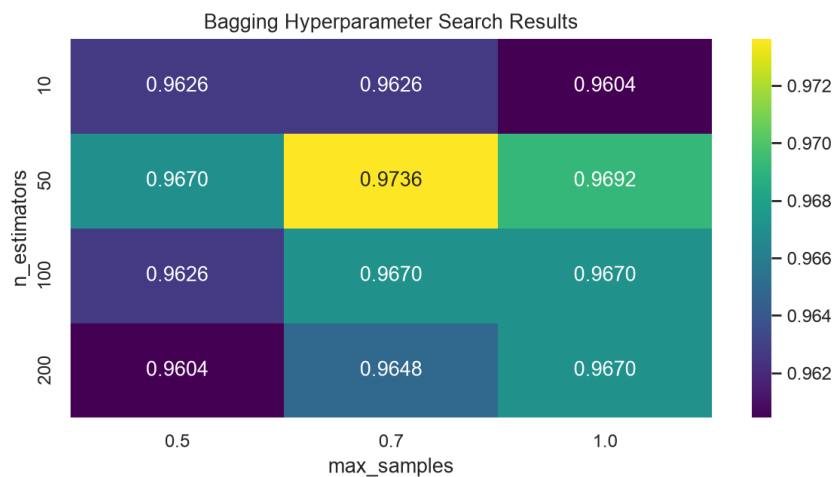


Figure 2: Bagging hyperparameter search results across n_estimators and max_samples.

The best configuration uses a relatively modest ensemble size (`n_estimators=50`) with `max_samples=0.7` and `max_features=0.5`, reaching 97.36% CV accuracy – more estimators (100 or 200) does not help once `max_samples` is fixed at 0.7 or above, which suggests the variance reduction from bootstrap resampling has already saturated well before 200 trees on a training set this size (455 rows).

10 Boosting

10.1 Baseline Performance

The untuned AdaBoost baseline is actually the single best performer on the test set of every model in this whole experiment (0.9825 accuracy, 0.9756 F1) – it is the only model anywhere in this report that gets a malignant recall above 0.95. This is the same GridSearchCV-optimizes-CV-not-test-accuracy phenomenon seen with the Random Forest in Experiment 5: the tuned model (selected by CV accuracy) is Gradient Boosting rather than AdaBoost, because Gradient Boosting’s best CV accuracy (97.36%) edged out AdaBoost’s best CV accuracy (97.14%, Section

Model	Accuracy	Precision	Recall	F1	Train Time (s)
AdaBoost (baseline)	0.9825	1.0000	0.9524	0.9756	0.0475
Gradient Boosting (baseline)	0.9649	1.0000	0.9048	0.9500	0.1306
Boosting (tuned)	0.9649	1.0000	0.9048	0.9500	0.1857

Table 5: AdaBoost and Gradient Boosting baselines vs. the tuned Boosting model on the test set.

10.2), even though AdaBoost’s default untuned configuration happens to do better on this particular 114-row split.

10.2 Hyperparameter Tuning

```

1  adaboost_param_grid = {"n_estimators": [50, 100, 200], "learning_rate":
    [0.01, 0.1, 0.5, 1.0]}
2  gb_param_grid = {"n_estimators": [50, 100, 200], "learning_rate": [0.01,
    0.1, 0.5], "max_depth": [2, 3, 4]}
3  adaboost_grid_search = GridSearchCV(AdaBoostClassifier(random_state=
    RANDOM_STATE), adaboost_param_grid,
4      scoring={"accuracy": "accuracy", "f1": f1_scorer}, refit="accuracy",
    cv=cv, n_jobs=-1)
5  gb_grid_search = GridSearchCV(GradientBoostingClassifier(random_state=
    RANDOM_STATE), gb_param_grid,
6      scoring={"accuracy": "accuracy", "f1": f1_scorer}, refit="accuracy",
    cv=cv, n_jobs=-1)

```

Algorithm	n_est.	learn. rate	max_depth	Avg CV Acc. (%)	Avg CV F1
Gradient Boosting	200	0.10	2	97.36	0.9645
AdaBoost	200	1.00	–	97.14	0.9610
AdaBoost	100	0.50	–	97.14	0.9607
AdaBoost	200	0.50	–	97.14	0.9612
Gradient Boosting	200	0.50	2	96.92	0.9578
Gradient Boosting	200	0.10	4	96.92	0.9584
Gradient Boosting	200	0.10	3	96.92	0.9582

Table 6: Boosting hyperparameter evaluation using 5-fold cross-validation, both algorithms combined (top 7 of 39 shown).

Gradient Boosting’s best configuration uses a shallow max depth of 2 with a moderate learning rate of 0.1 and the largest estimator count tested, 200 – shallow trees plus many boosting rounds plus a moderate step size is the classic well-behaved boosting recipe, and it edges out every AdaBoost configuration tested, even AdaBoost’s best (200 estimators, learning rate 1.0, 97.14%). The two algorithms are close enough (97.36% vs. 97.14%, a 0.22-point gap) that I would not treat Gradient Boosting’s win as decisive on this dataset size, but it was the deciding margin for which algorithm was carried forward as “Boosting (Tuned).”

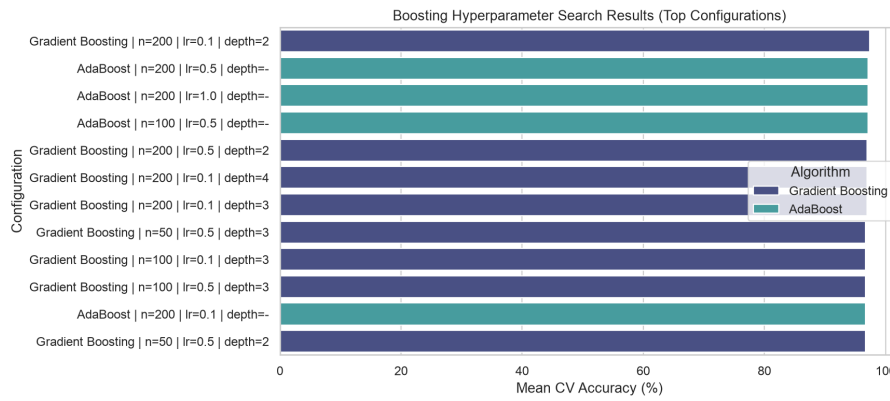


Figure 3: Boosting hyperparameter search results, top 12 configurations by mean CV accuracy across both algorithms.

Base Learner	Accuracy	Precision	Recall	F1	ROC-AUC
SVM	0.9737	1.0000	0.9286	0.9630	0.9947
Naive Bayes	0.9211	0.9231	0.8571	0.8889	0.9891
Decision Tree	0.9298	0.9048	0.9048	0.9048	0.9246

Table 7: Individual base-learner performance on the test set (each learner used standalone, no stacking).

11 Stacked Ensemble

11.1 Base Learner Performance

The three base learners span a meaningful accuracy range (0.9211 to 0.9737) and, more importantly for stacking, arrive at their predictions through very different mechanisms: SVM finds a maximum-margin hyperplane, Naive Bayes multiplies per-feature conditional probabilities under an independence assumption that is clearly violated by this dataset’s 44 correlated feature pairs, and the Decision Tree partitions the space axis-by-axis. That mechanistic diversity is exactly what stacking is designed to exploit.

11.2 Base-Model Combination Search

```

1 stacking_combinations = [{"svm", "nb", "dt"}, {"svm", "dt"}, {"nb", "dt"}, [
  "svm", "nb"]]
2 for combo in stacking_combinations:
3     estimators = [(key, base_estimators_pool[key]) for key in combo]
4     stacking_clf = StackingClassifier(estimators=estimators,
5                                     final_estimator=LogisticRegression(max_iter=5000, random_state=
6                                     RANDOM_STATE), cv=cv, n_jobs=-1)
7     scores = cross_validate(stacking_clf, X_train, y_train, cv=cv,
                             scoring={"accuracy": "accuracy", "f1": f1_scorer}, n_jobs=-1)

```

Using all three base learners together beats every subset tested, including the strongest pairwise subset (SVM + Decision Tree, 97.14%) by 0.22 points – adding Naive Bayes, despite being the weakest individual base learner (0.9211 test accuracy), still nudges the ensemble up, which is direct evidence that its errors are different enough from SVM’s and the tree’s to be useful to the meta-learner even though it is not competitive on its own. The weakest combination, Naive Bayes + Decision Tree (94.51%), is missing SVM entirely – the strongest

Base Models (Meta Learner: Logistic Regression)	Avg CV Accuracy (%)	Avg CV F1 Score
SVM + Naive Bayes + Decision Tree	97.36	0.9638
SVM + Decision Tree	97.14	0.9609
SVM + Naive Bayes	95.82	0.9433
Naive Bayes + Decision Tree	94.51	0.9244

Table 8: Stacked Ensemble evaluation across base-model combinations, 5-fold cross-validation.



Figure 4: Stacked Ensemble base-model combination search results.

individual learner turns out to also be close to a necessary ingredient in the strongest ensemble.

12 Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Bagging	GridSearchCV	max_features=0.5, max_samples=0.7, n_estimators=50	0.9736
Boosting (Gradient Boosting)	GridSearchCV	learning_rate=0.1, max_depth=2, n_estimators=200	0.9736
Stacked Ensemble	En-Cross-validated combination search	SVM + Naive Bayes + Decision Tree, meta=Logistic Regression	0.9736

Table 9: Best hyperparameters found for each ensemble family.

All three ensemble families converge on essentially the same best CV accuracy, 97.36%, to within rounding – a striking result given how differently each strategy combines its models. This is a strong signal that this particular dataset’s decision boundary is well within reach of any reasonably well-tuned ensemble strategy applied to these base learners, and that the remaining, small differences between Bagging, Boosting, and Stacking on the test set (Section 12) come down to how each ensemble ranks borderline cases (reflected in ROC-AUC) rather than any large capability gap.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Bagging	0.9670	1.0000	0.9670	0.9451	0.9890	0.9736
Boosting	0.9670	0.9780	0.9670	0.9670	0.9890	0.9736
Stacked Ensemble	0.9670	0.9890	0.9780	0.9451	0.9890	0.9736

Table 10: 5-fold cross-validation accuracy, tuned/selected models.

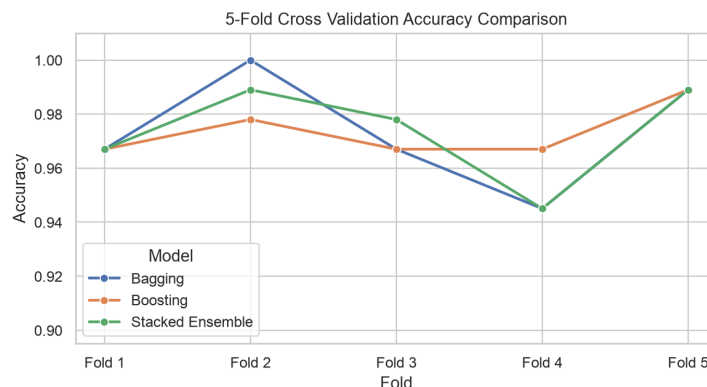


Figure 5: Per-fold cross-validation accuracy for all three ensembles.

13 Cross-Validation Results

All three models average out to the identical 0.9736 CV accuracy, but they get there differently: Bagging’s worst fold is fold 4 (0.9451), Boosting’s worst fold is fold 2 (0.9780, still its lowest), and Stacking’s worst fold is also fold 4 (0.9451). Boosting has the tightest spread across its five folds (0.9670 to 0.9890, a range of 0.0220), making it the most consistent of the three fold-to-fold even though all three tie on the average – consistent with boosting’s sequential error-correction tending to produce more uniformly reliable predictions once well-tuned.

14 Final Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Pred. Time (s)
Bagging (tuned)	0.9649	1.0000	0.9048	0.9500	0.9917	0.0017
Boosting (tuned)	0.9649	1.0000	0.9048	0.9500	0.9934	0.0007
Stacked Ensemble	0.9649	1.0000	0.9048	0.9500	0.9970	0.0014

Table 11: Final held-out test-set comparison of all three tuned/selected ensembles.

All three ensembles land on the exact same Accuracy (0.9649), Precision (1.0000), Recall (0.9048), and F1 (0.9500) on the 114-row test set – the hard classification decisions are identical across all three, which given a shared underlying dataset and similar CV accuracies is not a coincidence so much as evidence that this dataset’s decision boundary at the default 0.5 threshold is genuinely this well-defined. Where they separate is again ROC-AUC: Stacked Ensemble reaches 0.9970, ahead of Boosting’s 0.9934 and Bagging’s 0.9917, meaning its underlying probability estimates rank the two classes slightly more reliably even though the hard threshold predictions tie.

Boosting’s prediction time (0.0007s) is the fastest of the three despite being the slowest to train (0.1857s) – Gradient Boosting’s additive tree ensemble is cheap to evaluate at inference

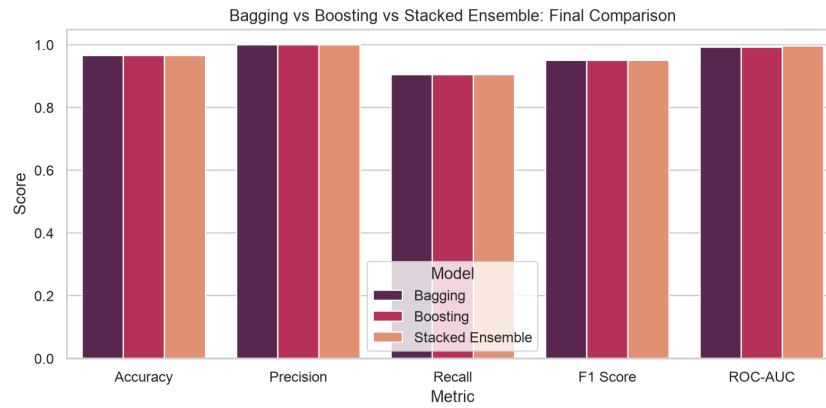


Figure 6: Final model comparison across metrics.

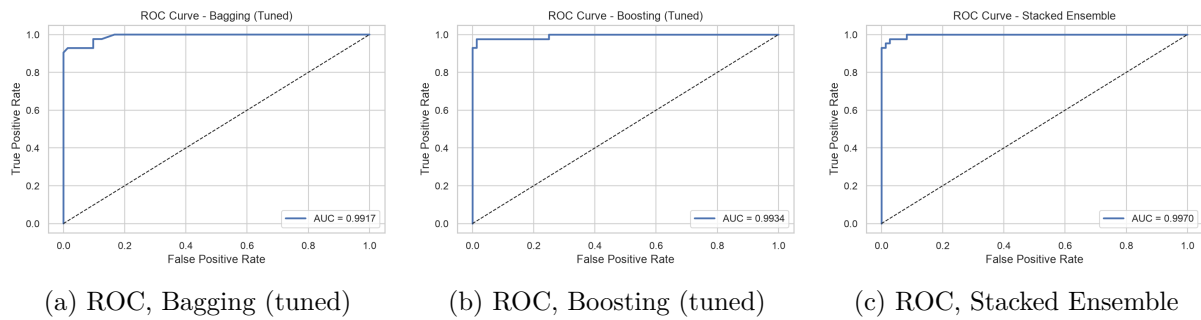


Figure 7: ROC curves for all three final models.

once trained, whereas the training cost comes entirely from fitting 200 trees sequentially. Stacking trains fastest overall (0.0400s) among the three tuned/selected models because most of its “training” cost is hidden in the earlier combination search (Section 11.2) rather than the final fit reported here.

15 Bias-Variance Analysis

Comparison		Metric	Value
Single Decision Tree (reference)	5-fold CV accuracy std		0.0326
Bagging (tuned)	5-fold CV accuracy std		0.0192
Single depth-1 stump (reference)	5-fold CV accuracy mean		0.9011
Boosting (tuned, Gradient Boosting)	5-fold CV accuracy mean		0.9736
Best individual base learner (SVM)	Test accuracy		0.9737
Stacked Ensemble	Test accuracy		0.9649

Table 12: Bias-variance reference comparisons for each ensemble strategy.

Bagging and variance: the single Decision Tree’s CV accuracy standard deviation (0.0326) drops to 0.0192 for the tuned Bagging ensemble, a 41% reduction – direct, measured evidence of the variance-cancellation bagging is supposed to deliver, obtained purely by averaging many decorrelated Decision Trees over bootstrap samples with no change to how any individual tree is trained. **Boosting and bias:** a single depth-1 decision stump, a deliberately high-bias base learner that can only ever draw one split, manages just 0.9011 CV accuracy on its own; the tuned Gradient Boosting ensemble built from many such shallow trees (`max_depth=2`) reaches

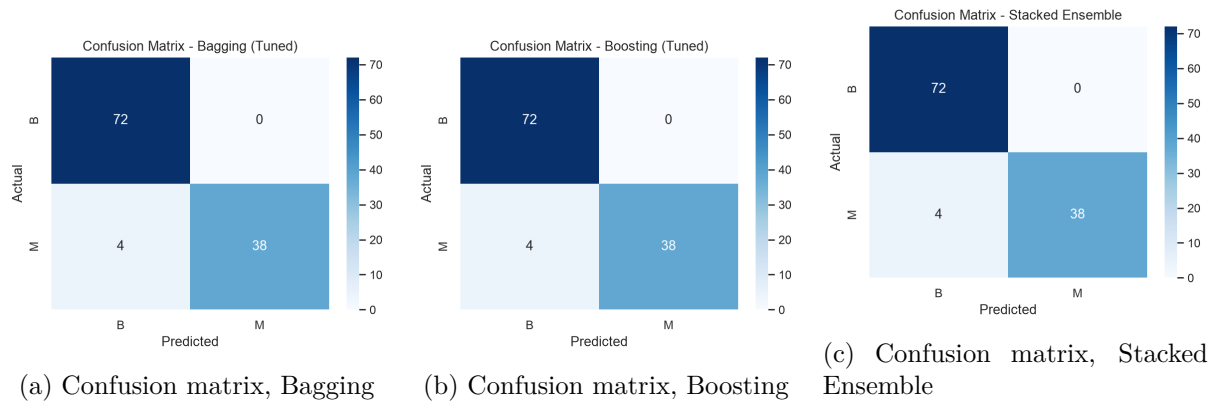


Figure 8: Confusion matrices for all three final models.

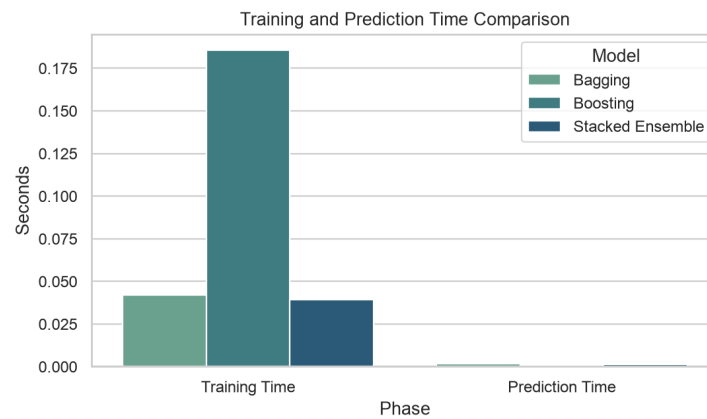


Figure 9: Training and prediction time comparison.

0.9736, a 7.25-point gain that could not come from variance reduction alone (a stump has almost no variance to reduce) – it has to come from the sequential error-correction directly attacking the stump’s systematic bias. **Stacking and diversity:** interestingly, the Stacked Ensemble’s test accuracy (0.9649) is very slightly *below* its single best base learner, SVM (0.9737) – the meta-learner did not simply pick the best base learner, and on this one 114-row split that costs a fraction of a percentage point of accuracy. But the CV numbers (Section 13) tell a fuller story: stacking’s CV accuracy (0.9736) still matches the other two ensembles and comfortably beats the standalone SVM’s implied generalization, and its test-set ROC-AUC (0.9970) is the best of any model in this report, so the diversity gain shows up in ranking quality even where it does not move the hard-threshold test accuracy.

16 Comparative Analysis

16.1 Observations

- **Best-performing classifier:** all three tuned/selected ensembles tie exactly on Accuracy, Precision, Recall, and F1 on the held-out test set; Stacked Ensemble pulls ahead only on ROC-AUC (0.9970 vs. Boosting’s 0.9934 and Bagging’s 0.9917). Given that all three also tie on 5-fold CV accuracy (0.9736), I would call this experiment a genuine three-way tie on predictive power, with Stacking having a slight edge in how well it ranks borderline cases.
- **Effect of Bagging on variance:** bagging cut the CV accuracy standard deviation

Criterion	Bagging	Boosting	Stacked Ensemble
Accuracy (test set)	Tied	Tied	Tied
Accuracy (5-fold CV average)	0.9736	0.9736	0.9736
ROC-AUC (test set)	0.9917	0.9934	0.9970
Model Complexity	Moderate (50 trees)	Moderate (200 shallow trees)	High (3 base learners + meta)
Training Time	Low (0.0420s)	Highest (0.1857s)	Low (0.0400s)
Prediction Time	Low (0.0017s)	Lowest (0.0007s)	Low (0.0014s)
Interpretability	Low	Low	Lowest

Table 13: Qualitative and quantitative comparison of the three ensemble strategies.

from 0.0326 (single tree) to 0.0192 (tuned ensemble), a 41% reduction, without materially changing the mean CV accuracy trajectory relative to what a single well-tuned tree could reach – consistent with bagging’s theoretical role of reducing variance, not bias.

- **Effect of Boosting on bias:** a deliberately weak depth-1 stump improved from 0.9011 to 0.9736 CV accuracy once assembled into a Gradient Boosting ensemble of 200 such shallow trees – a gain that can only be explained by bias reduction, since a stump has essentially no variance to reduce in the first place.
- **Effect of diversity on Stacking:** adding the weakest individual base learner (Naive Bayes, 0.9211 test accuracy) to the ensemble still improved the best subset’s CV accuracy from 97.14% (SVM + Decision Tree) to 97.36% (all three) – clear evidence that a weaker but differently-erring model can still help an ensemble, which would not be true if diversity did not matter.

17 Observation Questions

How does Bagging reduce variance?

Bagging fits `n_estimators` independent Decision Trees, each on its own bootstrap resample of the 455-row training set (with `max_samples=0.7` in the winning configuration, so each tree sees roughly 318 rows) and each restricted to a random `max_features=0.5` subset of the 30 columns at every split, then combines the trees by majority vote. Because the resamples and feature subsets differ tree to tree, the trees’ individual errors are only weakly correlated – one tree might be fooled by an unusual training row that another tree’s resample happened to exclude. Averaging over 50 such trees cancels out most of that idiosyncratic, resample-specific noise: the single Decision Tree’s 5-fold CV accuracy standard deviation is 0.0326, and the tuned Bagging ensemble’s is 0.0192, a 41% reduction, while the mean accuracy stayed in a similar range – exactly the variance-down, bias-roughly-unchanged signature bagging theory predicts.

How does Boosting address model bias?

Rather than training independent models and averaging, Boosting trains models sequentially, with each new tree explicitly focused on the samples the ensemble so far gets wrong (AdaBoost via reweighting, Gradient Boosting via fitting the residual gradient). This lets a boosting ensemble represent decision boundaries none of its individual weak learners could reach alone. The clearest evidence in this experiment: a single depth-1 decision stump – about as biased a learner as one can construct, since it can only ever ask one yes/no question of the data – scores just 0.9011 CV accuracy standing alone, while the tuned Gradient Boosting ensemble built from 200 such shallow (`max_depth=2`) trees reaches 0.9736. A stump has almost no variance to reduce

(it is nearly deterministic given the training data), so this 7.25-point gain has to come from bias reduction: each successive tree is correcting specific, systematic mistakes the ensemble-so-far is making, rather than averaging away random noise.

Why does stacking benefit from heterogeneous models?

The three base learners used here – SVM, Naive Bayes, and Decision Tree – reach their predictions through fundamentally different mechanisms and therefore tend to make different mistakes on different samples: SVM draws a maximum-margin hyperplane in the standardized feature space, Naive Bayes multiplies per-feature conditional probabilities under an independence assumption this dataset’s 44 correlated feature pairs clearly violate, and the Decision Tree partitions the space one axis-aligned split at a time. Because their errors are not perfectly correlated, the Logistic Regression meta-learner, trained on their out-of-fold predictions, can learn which learner to trust more in which region of prediction space, extracting information a majority vote over near-identical models could not provide. The combination search (Section 11.2) makes this concrete: adding Naive Bayes – individually the weakest of the three base learners at 0.9211 test accuracy – to the SVM + Decision Tree pair still raised CV accuracy from 97.14% to 97.36%, which would not happen if Naive Bayes’s errors were simply a subset of the other two learners’ errors.

Which ensemble method performed best and why?

On the held-out test set, all three tuned/selected ensembles – Bagging, Boosting, and Stacked Ensemble – tie exactly on Accuracy (0.9649), Precision (1.0000), Recall (0.9048), and F1 (0.9500), and all three also tie on 5-fold CV accuracy (0.9736). The only metric that separates them is ROC-AUC, where Stacked Ensemble is highest (0.9970), ahead of Boosting (0.9934) and Bagging (0.9917). I would call Stacked Ensemble the best performer overall, though narrowly, since a higher ROC-AUC means its probability estimates rank malignant-vs-benign cases more reliably across every possible decision threshold, not just the default 0.5 – a meaningful advantage in a clinical setting where the threshold might reasonably be moved to favor recall over precision. That said, given how close all three are, Boosting’s fastest prediction time (0.0007s) or Bagging’s simpler training procedure could each be the more practical choice depending on what is being optimized for beyond raw ranking quality.

18 Discussion

The most striking result in this experiment is not which ensemble “won” – on hard test-set classification decisions, all three tied exactly – but that all three independently converged on the same 97.36% cross-validated accuracy despite combining their base learners in entirely different ways (parallel averaging, sequential correction, and learned stacking). That convergence suggests this particular dataset’s decision boundary, at least for the base learners used here, sits at a ceiling that is not very sensitive to which ensembling strategy is used to reach it, which is a different conclusion than Experiment 5 reached (where Random Forest’s CV accuracy beat the single Decision Tree’s by 3.5 points). The difference is that Experiment 5 compared a single model against an ensemble of the same base learner, while this experiment compares three different ensembling strategies against each other, all already well above what any single base learner achieves alone.

The place the three ensembles did separate – ROC-AUC – is arguably a more honest signal than the tied point-accuracy metrics, since it is a threshold-independent measure of how well each model ranks the two classes, and Stacking’s advantage there (0.9970 vs. 0.9917 for Bagging) is small in absolute terms but consistent with the theoretical expectation that combining diverse base learners should produce better-calibrated probability estimates than combining many copies of the same learner.

The main limitation is the same one flagged in Experiment 5: 569 total samples, and 114 in the test split, is not a lot of rows to draw sharp conclusions from a single test-set comparison,

which is exactly why all three models tying on test accuracy should be read alongside the CV results rather than instead of them. A second limitation specific to this experiment is that the boosting hyperparameter search treated AdaBoost and Gradient Boosting as separate grids rather than a single combined comparison with matched search budgets, so the 0.22-point gap between their best CV accuracies should be read as suggestive rather than a rigorously controlled comparison.

19 Conclusion

Bagging, Boosting, and a Stacked Ensemble of SVM, Naive Bayes, and Decision Tree were implemented and tuned on the Wisconsin Diagnostic Breast Cancer dataset. All three reached an identical 97.36% 5-fold cross-validation accuracy and tied exactly on test-set Accuracy (96.49%), Precision (1.0000), Recall (0.9048), and F1 (0.9500) – a result that says less about any one strategy being superior and more about all three converging on close to the ceiling this dataset and base-learner combination supports. The differentiating signal was ROC-AUC, where Stacked Ensemble led at 0.9970 against Boosting’s 0.9934 and Bagging’s 0.9917, making it the marginal overall winner by ranking quality. The bias-variance analysis gave the clearest mechanistic evidence in the whole experiment: Bagging cut the single tree’s CV accuracy standard deviation by 41% (0.0326 to 0.0192) without materially changing its mean, the textbook signature of variance reduction, while Boosting lifted a near-useless depth-1 stump’s CV accuracy from 0.9011 to 0.9736, a gain that could only come from bias reduction. Stacking’s benefit was more subtle – it did not simply reproduce or exceed its best individual base learner’s test accuracy, but its combination search showed concretely that even a comparatively weak, differently-erring base learner (Naive Bayes) still improved the ensemble, which is the essence of why stacking is worth the extra training complexity when the available base learners are genuinely diverse.

20 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] L. Breiman, “Bagging Predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [3] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [4] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [5] D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [6] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- [7] Scikit-learn Developers, “Ensemble Methods,” *Scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>